

Imię i nazwisko	Wiktoria Sakowska (274931)
Nazwa uczelni	Uniwersytet Gdański
Nazwa wydziału	Matematyki, Fizyki i Informatyki
Kierunek	Informatyka Ogólnoakademicka
Prowadząca	mgr Laura Grzonka
Nazwa ćwiczenia	Projekt 2
Data zajęć	01.04.2026 r.
Data oddania	22.04.2026 r.

AstroApp

Dokumentacja projektu

Programowanie w języku Python — Projekt II (Flask)

Spis treści

1	Opis projektu	3
2	Wymagania i instalacja	3
2.1	Wymagania systemowe	3
2.2	Instalacja krok po kroku	3
2.3	Dostępne adresy	4
3	Architektura projektu	4
3.1	Struktura katalogów	4
3.2	Blueprinty i ich odpowiedzialności	5
3.3	Stack technologiczny	6
4	Modele danych	6
4.1	Diagram relacji	6
4.2	Opis modeli	7
5	Kluczowe funkcjonalności	7
5.1	Serwis astrologiczny (astro_service.py)	7
5.2	Wizualizacja SVG	7
5.3	Interpretacja AI (ai_service.py)	8
5.4	Konta użytkowników	8
5.5	CRUD chartów	8
5.6	Wyszukiwarka	8
5.7	REST API	9
6	Testy	9
7	Spełnienie wymagań projektowych	10
7.1	Elementy niezbędne	10
7.2	Opcje dodatkowe (6 z wymaganych 3)	10
8	Podział pracy	10
8.1	Praca samodzielna	10
8.2	Z pomocą AI	10
8.3	Z dokumentacji i samouczków	11
9	Bibliografia	11
9.1	Sztuczna inteligencja	11
9.2	Frameworki i biblioteki	11
9.3	Dokumentacja i źródła internetowe	11

1 Opis projektu

AstroApp to aplikacja webowa stworzona we frameworku Flask, która umożliwia generowanie map urodzeniowych (natal charts), porównywanie synastrii oraz uzyskiwanie interpretacji astrologicznych wspieranych przez sztuczną inteligencję.

Główne założenia aplikacji:

- Użytkownik podaje datę, godzinę i miejsce urodzenia — aplikacja oblicza pozycje planet, ascendent i domy astrologiczne.
- Wizualizacja mapy urodzeniowej w formie interaktywnego SVG (koło zodiaku z planetami i aspektami).
- Synastria — porównanie dwóch map urodzeniowych z wizualizacją i tabelą porównawczą.
- Interpretacja AI — Google Gemini analizuje chart i generuje spersonalizowany opis osobowości.
- Konta użytkowników z możliwością zapisywania, edycji i usuwania chartów.
- REST API umożliwiający generowanie chartów z poziomu zewnętrznych aplikacji.

2 Wymagania i instalacja

2.1 Wymagania systemowe

- Python 3.10 lub nowszy
- pip
- Git

2.2 Instalacja krok po kroku

```
# 1. Klonowanie repozytorium
git clone <URL_REPOZYTORIUM>
cd Project02

# 2. Srodowisko wirtualne
python3 -m venv venv
source venv/bin/activate

# 3. Instalacja zaleznosci
pip install -r requirements.txt

# 4. Konfiguracja klucza API (opcjonalne)
export GEMINI_API_KEY="twoj-klucz-z-aistudio.google.com"

# 5. Migracje bazy danych
export FLASK_APP=run.py
flask db init
flask db migrate -m "Initial"
flask db upgrade
```

```
# 6. Uruchomienie serwera
python run.py
```

2.3 Dostępne adresy

Adres	Opis
/	Strona główna
/charts/generate	Generowanie chartu
/charts/synastri	Synastria
/charts/history	Historia chartów
/search	Wyszukiwarka
/auth/register	Rejestracja
/auth/login	Logowanie
/auth/profile	Profil użytkownika
/about	O aplikacji
/api/chart	REST API — generowanie (POST)
/api/charts	REST API — lista chartów (GET)
/api/cities	REST API — lista miast (GET)

3 Architektura projektu

3.1 Struktura katalogów

Projekt wykorzystuje wzorzec *Application Factory* z podziałem na 4 blueprinty:

```
Project02/
|-- run.py                # Entry point
|-- config.py            # Config + TestConfig
|-- requirements.txt
|-- README.md, SOURCES.md
|-- app/
|   |-- __init__.py      # create_app (Application Factory)
|   |-- extensions.py    # db, migrate, login_manager, csrf
|   |-- models.py       # User, BirthChart
|   |-- main/           # Blueprint: strona glowna, about,
|       search
|   |   |-- routes.py
|   |   '-- templates/main/
|   |-- auth/           # Blueprint: rejestracja, logowanie,
|       profil
|   |   |-- routes.py, forms.py
|   |   '-- templates/auth/
```

```
| |-- charts/                                # Blueprint: generowanie, synastria,
CRUD, AI
| | |-- routes.py, forms.py
| | |-- astro_service.py
| | |-- ai_service.py
| | |-- templates/charts/
| |-- api/                                # Blueprint: REST API
| | |-- routes.py
| |-- templates/                          # base.html, 404.html, 500.html
| |-- static/css/
|-- tests/
| |-- conftest.py
| |-- test_models.py
| |-- test_auth.py
| |-- test_charts.py
|-- docs/                                # Dokumentacja LaTeX
```

3.2 Blueprinty i ich odpowiedzialności

Blueprint	Odpowiedzialność
main	Strona główna, o aplikacji, wyszukiwarka chartów
auth	Rejestracja, logowanie, wylogowanie, profil użytkownika
charts	Generowanie natal chartów, synastria, CRUD (historia, edycja, usuwanie), wizualizacja SVG, interpretacja AI
api	REST API — endpointy JSON do generowania chartów i pobierania danych

3.3 Stack technologiczny

Technologia	Wersja	Zastosowanie
Python	3.10+	Język programowania
Flask	3.1.3	Framework webowy
SQLAlchemy	via Flask-SQLAlchemy	ORM, modele danych
Flask-Migrate	4.1.0	Migracje bazy danych (Alembic)
Flask-Login	0.6.3	Autoryzacja użytkowników
Flask-WTF	1.2.2	Formularze, ochrona CSRF
kerykeion	5.12.7	Obliczenia astrologiczne (Swiss Ephemeris)
google-genai	1.73.1	Interpretacja AI (Gemini 2.5 Flash)
Bootstrap	5.3.3	Framework CSS (CDN, dark mode)
Bootstrap Icons	1.11.3	Ikony (CDN)
SQLite	3	Baza danych
pytest	8.3.5	Testy jednostkowe
python-dotenv	1.1.0	Zmienne środowiskowe (.env)

4 Modele danych

Projekt wykorzystuje **2 modele** z relacją 1:M (jeden użytkownik — wiele chartów).

4.1 Diagram relacji

```
User ---- 1:M ----> BirthChart
|
| username          | name, birth_date, birth_time
| email            | birth_city, sun_sign
| password_hash    | moon_sign, ascendant
| is_admin         | chart_data (JSON)
| created_at       | created_at, updated_at
```

4.2 Opis modeli

Model	Kluczowe pola	Cechy
User	username, email, password_hash, is_admin, created_at	UserMixin (Flask-Login), hashowanie hasła (Werkzeug), relacja 1:M do BirthChart z cascade delete
BirthChart	name, birth_date, birth_time, birth_city, sun_sign, moon_sign, ascendant, chart_data (JSON)	FK do User, przechowuje pełne dane planet w polu JSON

5 Kluczowe funkcjonalności

5.1 Serwis astrologiczny (astro_service.py)

Wrapper na bibliotekę kerykeion z polskimi tłumaczeniami. Zawiera:

- Słownik 25+ polskich miast ze współrzędnymi geograficznymi
- Tłumaczenia znaków zodiaku, planet i domów na język polski
- Funkcje: `generate_chart()`, `generate_chart_svg()`, `generate_synastry_svg()`

Fragment kodu generowania chartu:

```
1 subject = AstrologicalSubject(  
2     name, year, month, day, hour, minute,  
3     city, country,  
4     lng=lng, lat=lat, tz_str="Europe/Warsaw",  
5 )  
6 # Zbieramy pozycje planet  
7 for attr in ["sun", "moon", "mercury", ...]:  
8     p = getattr(subject, attr)  
9     planets.append({  
10         "name_pl": PLANET_NAMES_PL[p.name],  
11         "sign": SIGN_NAMES_PL[p.sign],  
12         "position": round(p.position, 1),  
13         "house": HOUSE_NAMES_PL[p.house],  
14     })
```

5.2 Wizualizacja SVG

Mapy urodzeniowe i synastrie generowane są jako SVG przez kerykeion (`KerykeionChartSVG`). Ze względu na rozmiar SVG (220 KB), nie jest ono przechowywane w sesji cookie (limit 4 KB), lecz generowane dynamicznie w dedykowanym endpointzie `/charts/svg` i ładowane przez tag `<object>`.

5.3 Interpretacja AI (ai_service.py)

Integracja z Google Gemini API (gemini-2.5-flash):

```
1 from google import genai
2
3 client = genai.Client(api_key=api_key)
4 response = client.models.generate_content(
5     model="gemini-2.5-flash",
6     contents=prompt,
7 )
```

Prompt zawiera pozycje planet i prosi model o interpretację w 5 obszarach: osobowość, emocje, komunikacja, relacje, wyzwania. Klucz API przechowywany w zmiennej środowiskowej GEMINI_API_KEY.

Aplikacja działa poprawnie bez klucza — interpretacja AI jest wtedy niedostępna, ale wszystkie inne funkcje działają normalnie.

5.4 Konta użytkowników

- Rejestracja z walidacją unikalności nazwy użytkownika i emaila
- Logowanie z opcją „zapamiętaj mnie” (Flask-Login)
- Profil użytkownika z listą zapisanych chartów
- Hashowanie haseł (Werkzeug: `generate_password_hash`)

5.5 CRUD chartów

- **Create** — generowanie chartu + zapisywanie do bazy
- **Read** — historia chartów, podgląd z tabelą planet i mapą SVG
- **Update** — edycja danych urodzeniowych z przeliczeniem chartu
- **Delete** — usuwanie z potwierdzeniem (JavaScript confirm)

5.6 Wyszukiwarka

Przeszukiwanie chartów po imieniu, znaku słonecznym, księżycowym, ascendencie i mieście. Wykorzystuje `ilike` (case-insensitive). Dodatkowe szybkie filtry — przyciski 12 znaków zodiaku.

5.7 REST API

Metoda	Endpoint	Opis
POST	/api/chart	Generuje chart z JSON body i zwraca pozycje planet
GET	/api/charts	Lista chartów zalogowanego użytkownika
GET	/api/charts/<id>	Szczegóły chartu z danymi planet
GET	/api/cities	Lista obsługiwanych miast

Blueprint API jest wyłączony z ochrony CSRF (`csrf.exempt`) — klienty REST nie wysyłają tokenów CSRF.

6 Testy

Projekt zawiera **34 testy** uruchamiane komendą:

```
python -m pytest tests/ -v
```

Obszar	Testowane elementy	Liczba
Modele	User (tworzenie, hasło, repr), BirthChart (tworzenie, relacje, cascade)	7
Autoryzacja	Rejestracja, duplikat, logowanie, złe hasło, profil	8
Serwis astro	Generowanie chartu, nieznane miasto, współrzędne, znaki	6
Widoki chartów	Formularz, POST, wynik, sesja, synastria, SVG	6
CRUD	Zapisywanie, historia, usuwanie	3
REST API	POST chart, brakujące pola, lista miast, autoryzacja	4
Razem		34

Testy wykorzystują fixture'y pytest (`conftest.py`): `app`, `db` (czysta baza SQLite in-memory), `client`, `user`, `logged_client`.

7 Spełnienie wymagań projektowych

7.1 Elementy niezbędne

Wymaganie	Realizacja
Min. 5 podstron	12+ podstron (home, about, generate, result, synastry, history, view, edit, search, profile, login, register)
Komunikacja z bazą (CRUD)	Pełny CRUD chartów urodzeniowych
Min. 5 widoków	20+ widoków (routes w 4 blueprintach)
Blueprinty	4 blueprinty: main, auth, charts, api
Min. 1 formularz	4 formularze: BirthDataForm, SynastryForm, RegistrationForm, LoginForm
Obsługa 404 i 500	Własne szablony z motywem astrologicznym
Testy	34 testy (pytest)

7.2 Opcje dodatkowe (6 z wymaganych 3)

1. **Stylizacja CSS / framework** — Bootstrap 5.3 (dark mode) + Bootstrap Icons + własny `style.css`
2. **Jinja2** — dziedziczenie szablonów, filtry, makra
3. **REST API** — 4 endpointy JSON (generowanie, lista, szczegóły, miasta)
4. **Konta użytkowników + logowanie** — rejestracja, logowanie, profil, rola admin
5. **Chat AI** — interpretacja chartu i synastrii przez Google Gemini
6. **Przeszukiwanie strony** — wyszukiwarka chartów po imieniu, znaku, mieście

8 Podział pracy

8.1 Praca samodzielna

- Koncepcja i projekt aplikacji — wybór tematu (astrologia), zaplanowanie funkcjonalności
- Projektowanie modeli danych i relacji
- Serwis astrologiczny — wrapper na kerykeion, polskie tłumaczenia, słownik miast
- Konfiguracja blueprintów i application factory
- Formularze z walidacją (WTForms)
- Szablony HTML i dostosowanie stylów CSS
- Integracja komponentów i debugowanie
- Dokumentacja (README, SOURCES, niniejszy dokument)

8.2 Z pomocą AI

- Wsparcie przy pisaniu kodu — generowanie szkieletu widoków, szablonów, formularzy
- Debugowanie błędów (CSRF, sesja, kerykeion API)
- Pomoc przy strukturyzacji projektu i planowaniu commitów

- Testy jednostkowe — szkielet i konfiguracja pytest

8.3 Z dokumentacji i samouczków

- Oficjalna dokumentacja Flask — blueprinty, szablony, application factory
- Dokumentacja Flask-SQLAlchemy — modele, migracje
- Dokumentacja Flask-Login — autoryzacja
- Dokumentacja kerykeion — obliczenia astrologiczne, SVG
- Dokumentacja Google Gemini API — integracja modelu AI
- Dokumentacja Bootstrap 5 — klasy CSS, komponenty

9 Bibliografia

9.1 Sztuczna inteligencja

Narzędzie	Model	Firma	Zastosowanie	Dostęp
Claude	Claude Opus 4.6	Anthropic	Wsparcie przy kodowaniu, debugowanie, planowanie	IV 2026
Gemini	gemini-2.5-flash	Google	Interpretacja chartu i sygnalii (w aplikacji)	IV 2026

9.2 Frameworki i biblioteki

Nazwa	Wersja	Strona	Dostęp
Flask	3.1.3	https://flask.palletsprojects.com/	IV 2026
Flask-SQLAlchemy	3.1.1	https://flask-sqlalchemy.readthedocs.io/	IV 2026
Flask-Migrate	4.1.0	https://flask-migrate.readthedocs.io/	IV 2026
Flask-Login	0.6.3	https://flask-login.readthedocs.io/	IV 2026
Flask-WTF	1.2.2	https://flask-wtf.readthedocs.io/	IV 2026
kerykeion	5.12.7	https://github.com/g-battaglia/kerykeion	IV 2026
google-genai	1.73.1	https://pypi.org/project/google-genai/	IV 2026
Bootstrap	5.3.3	https://getbootstrap.com/	IV 2026
Bootstrap Icons	1.11.3	https://icons.getbootstrap.com/	IV 2026
pytest	8.3.5	https://docs.pytest.org/	IV 2026
python-dotenv	1.1.0	https://pypi.org/project/python-dotenv/	IV 2026

9.3 Dokumentacja i źródła internetowe

Tytuł	URL	Dostęp
Flask — Quickstart	https://flask.palletsprojects.com/en/stable/quickstart/	IV 2026
Flask — Blueprints	https://flask.palletsprojects.com/en/stable/blueprints/	IV 2026
Flask — Templates	https://flask.palletsprojects.com/en/stable/templating/	IV 2026
Flask — Testing	https://flask.palletsprojects.com/en/stable/testing/	IV 2026
Flask-SQLAlchemy — Models	https://flask-sqlalchemy.readthedocs.io/en/stable/models/	IV 2026
Flask-Login — How It Works	https://flask-login.readthedocs.io/en/latest/	IV 2026
Flask-WTF — Creating Forms	https://flask-wtf.readthedocs.io/en/stable/form.html	IV 2026
Jinja2 — Templates	https://jinja.palletsprojects.com/en/stable/templates/	IV 2026
kerykeion — Documentation	https://kerykeion.net/docs/	IV 2026
Google Gemini API — Quickstart	https://ai.google.dev/gemini-api/docs/quickstart	IV 2026
Bootstrap 5 — Docs	https://getbootstrap.com/docs/5.3/	IV 2026
Werkzeug — Security	https://werkzeug.palletsprojects.com/en/stable/ utils/	IV 2026
pytest — Getting Started	https://docs.pytest.org/en/stable/getting-started.html	IV 2026